

PALLADIUM BANK — DIMENSIONAL DATA MODEL

Written Explanation: Design Justification & Reasoning

1. DATA PROFILING & DIMENSION IDENTIFICATION

The Palladium Bank transaction dataset contains 15 fields per record. Through systematic profiling, these were classified into three groups: **facts** (measurable numeric values), **dimensions** (descriptive context), and **degenerate dimensions** (identifiers with no additional attributes of their own).

Column	Type	Classification & Rationale
Amount (NGN)	NUMERIC	FACT — measurable, additive across dimensions
Balance_After (NGN)	NUMERIC	FACT — measurable, semi-additive (avg meaningful, sum not)
Txn_ID	VARCHAR	DEGENERATE DIMENSION — unique ID with no attributes; stored in fact
Txn_Date	DATETIME	DIMENSION key → dim_date (extracted to date_key)
Customer_ID / Name / Tier	VARCHAR	DIMENSION → dim_customer (SCD Type 2)
Branch_ID / Branch_Name / State	VARCHAR	DIMENSION → dim_branch (SCD Type 1)
Product_ID / Product_Name / Type	VARCHAR	DIMENSION → dim_product (SCD Type 1)
Txn_Type	VARCHAR	DIMENSION → dim_transaction_type
Channel	VARCHAR	DIMENSION → dim_channel

Fact Table Grain Decision: The grain was set at **one row per individual transaction**. This was chosen deliberately over daily or monthly summaries because the business requirements explicitly include churn signal detection (requiring recency and frequency analysis at the transaction level) and channel behaviour analysis (requiring individual transaction-level detail). A coarser grain would make it impossible to compute last-transaction recency accurately without re-querying the raw source.

Degenerate Dimension: Txn_ID is retained in the fact table as a degenerate dimension. It is the source system's unique transaction identifier and is essential for lineage and de-duplication checks, but it has no descriptive attributes that would justify a separate dimension table. Storing it in the fact table eliminates a join while preserving auditability.

2. STAR SCHEMA DESIGN JUSTIFICATION

A **star schema** was chosen over a snowflake schema for this implementation. Star schemas denormalize dimension data into flat, wide tables, which reduces join complexity and improves query performance for BI tools. Given that Palladium Bank's dashboards need to answer ad-hoc analytical questions quickly — and the dimension tables are small enough that storage savings from normalization are negligible — the star schema is the optimal choice.

The schema consists of one central fact table and six supporting tables:

- **fact_transactions:** Core fact table containing all transactional measures and foreign keys.
- **dim_date:** Conformed date dimension enabling Year→Quarter→Month drill-down hierarchies.
- **dim_customer:** SCD Type 2 customer dimension capturing tier changes over time.
- **dim_product:** SCD Type 1 product dimension with deposit/withdrawal classification flags.
- **dim_branch:** SCD Type 1 branch dimension enabling State→Branch geographic hierarchy.
- **dim_channel:** Static channel dimension classifying physical vs digital channels.

- **dim_transaction_type**: Static dimension classifying transaction direction and fee generation.

3. SLOWLY CHANGING DIMENSIONS (SCD) STRATEGY

Dimension	SCD Type	Justification
dim_customer	Type 2 (Full History)	Customer tier changes are business-critical. Platinum→Gold demotion must be tracked his
dim_product	Type 1 (Overwrite)	Product names and classifications rarely change. When they do, historical accuracy of the
dim_branch	Type 1 (Overwrite)	Branch metadata (name, state) is highly stable. Regulatory reporting does not require histo

The SCD Type 2 implementation for dim_customer uses three control columns: **tier_start_date** (when this version became active), **tier_end_date** (NULL for current records), and **is_current** (Boolean flag). The is_current flag is redundant but critical for query performance — filtering on is_current = TRUE avoids a date range comparison on every join.

4. ETL / ELT STRATEGY

Initial Load (18 months of historical data):

Tables must be populated in dependency order to avoid foreign key violations. The sequence is: (1) dim_date → (2) dim_channel → (3) dim_transaction_type → (4) dim_product → (5) dim_branch → (6) dim_customer → (7) fact_transactions. Required transformations include: extracting date_key from transaction timestamps as YYYYMMDD integers, standardizing tier values, deriving fee_amount from txn_type and amount rules, computing txn_hour from the timestamp, and resolving natural keys (customer_id, product_id) to their surrogate keys before inserting fact rows.

Incremental Loads:

- **Detect new records:** Compare incoming txn_id values against existing fact table using a staging table and LEFT JOIN anti-pattern (WHERE fact.txn_id IS NULL).
- **Handle SCD updates:** For dim_customer, compare incoming tier against is_current=TRUE record. On mismatch, close the old row and insert a new version.
- **Prevent duplicate facts:** Implement a UNIQUE constraint on txn_id in the fact table. Use INSERT ... ON CONFLICT DO NOTHING for idempotent loads.

Data Quality Checks (at minimum 5 defined):

#	Check	Detection Method	Action if Failed
1	Referential integrity	LEFT JOIN fact → dim; find NULLs	Quarantine orphan records; alert ETL team
2	Zero/negative amount	WHERE txn_amount <= 0	Flag in DQ error table; exclude from aggregations
3	Duplicate transactions	GROUP BY txn_id HAVING COUNT > 1	Deduplicate using ROW_NUMBER(); keep first load
4	Invalid date keys	LEFT JOIN to dim_date; find missing keys	Populate missing dates in dim_date before loading facts
5	Negative balances on savings	WHERE balance_after < 0 AND product_type IN ('Account,Savings')	IR (Account, Savings) report for manual operations review

5. HIERARCHIES, PERFORMANCE & SCALABILITY

Drill-Down Hierarchies:

- **Time Hierarchy:** Fiscal_Year → Quarter → Month → Week → Day (defined in dim_date via year_number, quarter_label, month_number columns)
- **Geography Hierarchy:** State → Branch_Name (via dim_branch.state and dim_branch.branch_name)
- **Customer Hierarchy:** Tier → Customer_ID → Individual Transaction
- **Product Hierarchy:** Product_Type → Product_Name

Partitioning Strategy:

The fact_transactions table is partitioned by **month (txn_timestamp)** using PostgreSQL declarative range partitioning. With 18 months of data and potentially millions of transactions, month-based partitioning ensures incremental load jobs write only to the current month's partition, and monthly dashboard queries scan only the relevant partition rather than the full table. Each new month triggers creation of a new partition, making the design automatically scalable.

Indexing Approach:

Indexes are created on all foreign key columns in fact_transactions (date_key, customer_sk, product_sk, branch_sk, channel_sk, txn_type_sk) to accelerate dimension joins. Additionally, a composite index on (branch_sk, date_key) directly supports the most common dashboard query pattern: branch performance over a date range. The txn_timestamp column is indexed to support recency-based churn queries. Dimension surrogate key columns are indexed via their primary keys.

Aggregation Table — agg_monthly_branch_summary:

A pre-aggregated summary table is built monthly, rolling up fact_transactions by branch and month. This table stores total transaction counts, unique customers, total amounts, fee income, deposits, and withdrawals. BI dashboard queries for branch-level monthly reporting query this aggregate table (hundreds of rows) rather than the full fact table (potentially millions of rows), delivering sub-second dashboard load times. The table is refreshed incrementally at month-end using an INSERT ... ON CONFLICT DO UPDATE (upsert) pattern keyed on (year_month, branch_sk).